

Zedral — M2 Maintenance Intelligence

Developer Handover · v1 (preventive scope; predictive interfaces only)

Prepared for Z Company · Zedral platform

2026-06-01

Table of Contents

Compiled developer handover for **M2 · Maintenance Intelligence**. The individual source specs (00–10) and the runnable M2_schema.sql live alongside this file; the planning rationale is in ../M2_Maintenance_Intelligence_DeepPlan.md.

Zedral · M2 Maintenance Intelligence — Developer Handover

00 · Master Index, Architecture, Stack & Conventions

Audience: engineering team · **Status:** v1 for build (preventive scope; predictive interfaces only) · **Date:** 2026-06-01

This is the build-ready specification set for **M2 · Maintenance Intelligence**, the first Zedral *module* (the foundation layers — Data Lake, Canonical Model, Manifold, Unified Intelligence, Monitoring, Reporting, Financial Impact, Platform/Security — are specified in ../../Dev_Handover/). Read this index first: it gives module scope, the architecture, which **platform decisions (D1–D11)** bind M2, the **stack** M2 inherits, M2-specific conventions, and the **build roadmap**. The *why* behind every spec is the planning deep-plan one level up: ../M2_Maintenance_Intelligence_DeepPlan.md.

1. Document set & reading order

#	Spec	Build priority
00	This index — architecture, stack, conventions, roadmap	Read first
01	Data Model & Schema — new canonical entities,	Phase M2-0

	DDL dictionary (M2_schema.sql)	
02	Asset Registry & Failure Taxonomy — ISO 14224 hierarchy, criticality, FMECA	Phase M2-0
03	Preventive Maintenance Engine — triggers, scheduling, WO generation	Phase M2-1
04	Work-Order Lifecycle & Events — notification→order state machine, capture, M1 bootstrap	Phase M2-1
05	Spares / MRO & Maintenance Cost — catalogue, BOM, consumption, costing	Phase M2-2
06	KPIs, Analytics & Financial Impact — formulas + SQL, registry, M2→M4 reconciliation	Phase M2-2
07	APIs, Events & Integration — REST/OpenAPI, domain events, SAP PM connector mapping	Phase M2-1→2
08	Predictive (PdM) — Future-Scope Interfaces — OSA- CBM hooks, ML- readiness gate (no v1 build)	Phase M2-4 (gated)
09	Security, RBAC, NFRs &	Cross-cutting

	Acceptance — roles, multi-tenancy, test plan, acceptance	
10	Glossary — maintenance & reliability terms	Reference
—	M2_schema.sql	Runnable PostgreSQL DDL (paired with 01)

Companion material: the planning deep-plan
 (../M2_Maintenance_Intelligence_DeepPlan.md) + 8 flowcharts
 (../M2_diagram_*.mermaid). Platform specs referenced throughout live in
 ../../Dev_Handover/ (01 Data Lake, 02 Canonical Model, 03 Manifold, 04 UIL, 05
 Monitoring, 06 Reporting, 07 Financial Impact, 08 Platform/Security, 09 Glossary).

2. Module scope & non-goals

In scope (v1, preventive): asset reliability registry & failure taxonomy; maintenance work-order lifecycle (corrective + preventive); preventive-maintenance engine (time / meter / basic condition triggers); spares/MRO & maintenance cost; the maintenance KPI family priced by Financial Impact; live asset-health surfacing; ingestion from an existing CMMS/EAM via Manifold **or** first-party capture **or** bootstrap from M1 stoppages.

Future scope (interfaces only, build gated — D9): predictive maintenance (condition→prognostics→RUL→advisory, the OSA-CBM pipeline) and the ML models behind it. v1 ships the *data hooks and interface stubs* (spec 08) so this is a later switch-on, not a re-model.

Non-goals (EAM, not this module): asset depreciation, capital replacement planning, disposal accounting, full procurement/PO workflows. The schema keeps hooks (asset_profile.acquisition_value) for a later EAM module.

3. System context

flowchart LR

```

subgraph SRC["Maintenance data in (capture OR ingest)"]
  CAP["M2 first-party capture (PWA, offline-tolerant)"]
  CMMS["Existing CMMS/EAM (SAP PM / Maximo) via Manifold"]
  M1S["M1 stoppages (MECH/ELECT/UTILITY)"]
  SENS["PLC meters & sensors (TimescaleDB)"]
end
subgraph M2SVC["M2 services"]
  REG["Asset & Reliability Registry"]
  PME["PM Engine (trigger scan → WO)"]
  WOE["Work-Order service"]

```

```

    SPR["Spares / Cost service"]
    KPI["KPI / Analytics service"]
    PDM["Predictive (OSA-CBM) — FUTURE, stub"]
end
CANON["canon.* (Event Spine + maint.* schema)"]
SERVE["Serving zone / KPI mart / Redis live cache"]
subgraph CONS["Consumers"]
    UIL["Unified Intelligence (KPI registry)"]
    MON["Operational Monitoring"]
    FIN["Financial Impact"]
    REP["Reporting"]
    M4["M4 OEE (shared downtime)"]
end
CAP --> WOE
CMMS --> REG
M1S --> WOE
SENS --> PME
SENS -.future.-> PDM
REG --> PME --> WOE --> SPR
WOE --> KPI
PDM -.future.-> KPI
REG & PME & WOE & SPR & KPI --> CANON --> SERVE
SERVE --> UIL & MON & FIN & REP & M4

```

Golden rule (inherited): M2 reads canonical data through the Serving zone and writes its facts back through the canonical write-back API + maint.* schema. It never reads another module's database and never writes canon core tables it does not own. A breakdown is **one** canon.event downtime row — M2 classifies it, M4 aggregates it.

4. Binding decisions that constrain M2 (subset of D1-D11)

#	Decision	M2 build constraint
D1	Hybrid-ready, per-client	PM trigger-scan + capture must run on-prem edge (plants are often disconnected); containerized services, no cloud-only calls.
D2	Near-real-time (minutes), tunable	PM due-scan and condition-rule evaluation cadence default = minutes, per-tenant config; capture never blocks.
D7	Cost-rates client-owned, effective-dated, “rate as of”	Labour/part/downtime costing resolves the canon.cost_rate effective at the event date , never “today’s” rate.
D8	Logical tenant isolation default	tenant_id on every maint.* row; all queries tenant-scoped at the data-access layer.
D9	Rule-based now, ML later	PM/condition logic is rule/threshold driven in v1; predictive ML is reserved (spec 08), not built.
D11	Sector-neutral, exercise	Asset/failure model is generic; Hero Steels is a <i>seed</i>

extension namespace *template*, not the schema. Client specifics live in ext JSONB.

Full D1–D11 text: ../../Dev_Handover/00_INDEX_and_Architecture.md §3.

5. Technology stack (inherited + M2-relevant)

M2 adds **no new platform technology**; it uses the stack ratified in the platform handover. The components M2 leans on most:

Concern	Tech	M2 use
Canonical OLTP	PostgreSQL 15+ (maint schema)	all M2 master + transactional tables (M2_schema.sql)
Time-series	TimescaleDB	maint.meter_reading (and future sensor features)
KPI mart	ClickHouse / Postgres columnar	kpi_asset_daily rollups for UIL/Reporting
Event bus	Kafka / Redpanda	publish/consume maintenance domain events (spec 07)
Scheduler	Apache Airflow	the periodic PM due-scan + meter-threshold + KPI rollup jobs
Live cache	Redis	asset-health board, due/overdue PM counts (Monitoring)
API	REST + OpenAPI , gateway, OIDC	spec 07 endpoints
Auth/RBAC	Keycloak (OIDC)	M2 roles extend M1 (spec 09)
Backend	Python 3.11 + FastAPI	M2 services
Frontend	React + TS (PWA)	capture + planner UI; reuse M1 capture patterns
ML (later)	Feast + MLflow	reserved for PdM (spec 08), not built in v1

6. M2 build roadmap (epics)

flowchart TD

M0["M2-0 · Foundations
maint schema · asset_profile · maintainable_item · failure_mode · seed taxonomies · RBAC roles"]

M1["M2-1 · Capture & Execute
notification→work_order lifecycle · M1-stoppage bootstrap · PM engine (time/meter) · basic condition alerts · capture UI"]
M2x["M2-2 · Measure & Price
spares/MRO + cost · KPI engine (MTBF/MTTR/availability/PM-compliance) · Financial Impact tagging · KPI registry publish · monitoring board"]
M3["M2-3 · Integrate & Scale
Manifold SAP PM/Maximo connector + maintenance sector template · reporting pack"]
M4["M2-4 · Predictive (GATED on data + D9)
OSA-CBM services · anomaly → RUL → advisory · Feast/MLflow"]
M0 --> M1 --> M2x --> M3 --> M4

Maps onto the platform roadmap: M2-0→2 ride Phase 2–3 (serve/synthesize); M2-3 rides Phase 4 (SAP/connectors); M2-4 rides Phase 5 (ML, data-gated).

7. Non-functional baseline (delta over platform §7)

M2 inherits the platform NFR baseline (multi-tenant, hybrid, minutes-latency, 99.5% serving, OIDC, audit, OTel). Module-specific targets (detailed in spec 09): PM due-scan completes within its cadence for ≥100k active triggers/tenant; first-party capture is **offline-tolerant** (queue locally, sync on reconnect) like M1; KPI rollups reconcile with M4 to the downtime-hour.

8. Open items (carry into build)

- Confirm the v1 **failure-mode taxonomy**: reuse M1 stoppage codes (MECH/ELECT/UTILITY) vs a fuller ISO 14224 set (spec 02 §6 — open decision).
- **Meter source**: PLC-automatic vs manual entry for usage-based PM (spec 03).
- **CMMS connector priority**: SAP PM first (Hero Steels runs SAP) — confirm (spec 07).
- **Predictive data gate**: the history threshold that flips an asset to “predictive-ready” (spec 08).
- **D11**: whether M2 is the module used to validate the canonical model on a non-steel client.

Next: specs 01–10. The compiled Word/PDF package combines all of these into one M2 Developer Handover document.

01 · M2 Data Model & Schema — Developer Specification

Component owner: Data/Domain · **Phase**: M2-0 · **Depends on**: platform 02 Canonical Model, 08 Platform · **Consumed by**: all M2 specs · **Runnable DDL**: M2_schema.sql

1. Purpose & scope

Defines the data M2 owns and how it extends the canonical model. **In scope:** the new maint.* entities, their fields, keys, relationships, and how they map onto the canonical **Event Spine**. **Out of scope:** how external maintenance data is mapped in (Manifold — spec 07 connector section); KPI computation (spec 06).

2. Modeling rules (MUST)

- Follow canonical DDL conventions (platform spec 02 §5): **lower_snake_case**, surrogate *_id (BIGINT), tenant_id UUID on every row, TIMESTAMPTZ UTC, JSONB ext for client-specific fields, FK to canon.* — **never** free-text codes where a master exists.
- M2 tables live in the **maint** schema. M2 **MUST NOT** alter canon core tables; it references them and writes Event-Spine specializations through the canonical write-back API.
- Three entities already exist on the canonical Event Spine (platform 02 §3.2) — M2 **populates**, not redefines, them: maintenance_work_order, pm_schedule, failure_event. The detailed operational tables live in maint.* and link back via downtime_event_ref → canon.event(event_id).
- Every failure record uses the ISO 14224 quartet (mode / mechanism / cause / effect); every reading carries units.

3. Entity catalog (new in maint)

Group	Tables	Purpose
Asset reliability	asset_profile, maintainable_item	criticality + nameplate on a canon asset; ISO 14224 L7–L9 subtree
Failure taxonomy	failure_mode	FMECA-scored failure modes (the reliability language)
Usage	meter, meter_reading	usage counters for meter-based PM (TimescaleDB)
PM model	maintenance_plan, plan_trigger, maintenance_task, task_part	plans, their triggers, checklists, expected parts
Execution	notification, work_order, wo_task, wo_labor	problem reports → jobs → executed tasks + labour
Failures	failure_event	the failure occurrence (mirrors canon failure_event)
Spares/MRO	spare_part, spare_bom, wo_part, spare_txn	catalogue, asset BOM, consumption, stock ledger

Condition/PdM	condition_alert, asset_health	basic CBM alerts (v1) + future RUL/health slot (no v1 producer)
Analytics	kpi_asset_daily	materialized KPI rollup grain

Field-level detail is in M2_schema.sql (column comments included). Key generated columns: failure_mode.rpn = severity×occurrence×detectability;
failure_event.downtime_h = (up_ts – down_ts) in hours.

4. ERD (core)

erDiagram

```

EQUIPMENT_NODE ||--o| ASSET_PROFILE : "1:1 reliability ext"
EQUIPMENT_NODE ||--o{ MAINTAINABLE_ITEM : "contains (L7-L9)"
MAINTAINABLE_ITEM ||--o{ FAILURE_MODE : "exhibits"
EQUIPMENT_NODE ||--o{ METER : "metered by"
METER ||--o{ METER_READING : "records"
MAINTENANCE_PLAN ||--o{ PLAN_TRIGGER : "fires on"
MAINTENANCE_PLAN ||--o{ MAINTENANCE_TASK : "checklist"
MAINTENANCE_PLAN ||--o{ WORK_ORDER : "generates"
NOTIFICATION ||--o| WORK_ORDER : "converts to"
WORK_ORDER ||--o{ WO_TASK : "executes"
WORK_ORDER ||--o{ WO_LABOR : "consumes labour"
WORK_ORDER ||--o{ WO_PART : "consumes parts"
FAILURE_EVENT ||--o| WORK_ORDER : "triggers corrective"
FAILURE_MODE ||--o{ FAILURE_EVENT : "classifies"
SPARE_PART ||--o{ WO_PART : "issued as"
SPARE_PART ||--o{ SPARE_TXN : "moves"
EVENT ||--o| WORK_ORDER : "downtime_event_ref"

```

5. Mapping to the canonical Event Spine

maint record	Canonical write-back	Notes
work_order (corrective/PM)	canon.event (event_type='maintenance') + maintenance_work_order specialization	the operational detail stays in maint.work_order
failure_event	canon.event (event_type='failure') + links the same downtime_event_ref	one downtime row, shared with M4
plan_trigger next-due	pm_schedule specialization	exposes “due/overdue” to Monitoring
meter_reading	stays in maint (TimescaleDB); summarized to serving	feeds meter-based triggers

6. Data contract (the M2 row — drives Manifold readiness)

● Required · ○ Additional (full matrix: platform 02 \$6 / planning 01 \$15).

Canonical group	M2	Unlocks
Asset hierarchy (+ item/part)	●	the registry
Downtime + ReasonCode	●	failure/breakdown analytics
Maintenance WO / PM / Failure	●	the module itself
Failure mode / criticality	●	reliability analytics
Meter / MeterReading	○→●	required only for meter-based PM
SensorReading	○→●	required to unlock predictive (spec 08)
Shift, Personnel, Spares, CostRate	○	non-disruptive scheduling, labour, cost, financial impact

7. Identity, units, time, versioning

- **Identity:** surrogate *_id; ingested CMMS rows keep source_ref + resolve via Manifold source_key_map.
- **Units:** embedded in column name / uom column; runtime in hours, tonnage in MT (plant convention).
- **Time:** all *_ts are TIMESTAMPTZ UTC; bucket to shift/day for kpi_asset_daily.
- **Versioning:** additive only; new fields via ext JSONB or additive migration (Flyway/Alembic). Serving APIs versioned independently.

8. NFRs

- meter_reading partitioned/hypertabled by reading_ts; indexed (meter_id, reading_ts DESC).
- work_order indexed on (tenant_id, status) and a partial index on open due_ts for the PM scan.
- All maint.* queries tenant-scoped; no cross-tenant path.

9. Acceptance criteria

- **MUST** create all maint.* objects from M2_schema.sql against an existing canon schema with zero errors.
- **MUST** represent every failure with mode/mechanism/cause/effect and link to a single canonical downtime event.
- **MUST** keep client-specific attributes in ext (no core schema change per client) — exercised by a non-steel sample (D11).

- **SHOULD** round-trip a sample SAP PM export (equipment, notification, order) into maint.* without loss.

10. Build phasing

M2-0: schema + seed taxonomies (criticality bands, failure-mode starter set, strategy enums) + migrations. Validated by loading the Hero Steels seed template (spec 02 \$7) and a generic non-steel sample.

11. Risks

Over-fitting to steel (mitigate: ext + D11 sample); event-table growth from high-frequency meters (retention/partition via Data Lake); dual source of truth if CMMS ingest and first-party capture both write the same asset (mitigate: Manifold dedup/source-key map — spec 07).

02 • Asset Registry & Failure Taxonomy — Developer Specification

Component owner: Reliability/Domain · **Phase:** M2-0 · **Depends on:** 01 Data Model, platform 02 Canonical Model · **Consumed by:** 03 PM Engine, 04 Work Orders, 06 KPIs

1. Purpose & scope

The reliability backbone: a disciplined asset hierarchy down to the maintainable item, a criticality model, and a controlled failure-mode vocabulary (ISO 14224). Without this, MTBF/MTTR and failure Pareto are not sliceable. **In scope:** asset profile, maintainable-item subtree, criticality/FMECA, failure taxonomy, seed templates. **Out of scope:** PM scheduling (03), KPI math (06).

2. Key concepts (MUST)

- The canonical ISA-95 hierarchy stops at WORK_UNIT (= the asset). M2 extends **downward** with maintainable_item (ISO 14224 levels **L7 SUBUNIT** → **L8 MAINTAINABLE_ITEM** → **L9 PART**) via a self-referential subtree, **without** altering canon.equipment_node.
- Each asset gets a 1:1 asset_profile (criticality, nameplate, default strategy).
- Each significant failure is described by the ISO 14224 quartet and FMECA-scored.

3. Asset hierarchy mapping (ISO 14224 × ISA-95)

ISO 14224 level	Where it lives	Example (generic)	Example (Hero Steels)
L1–L5 industry→plant/uni	canon.equipment_ node	plant / area	CR complex / mill bay

t	(ENTERPRISE... WORK_CENTER)		
L6 equipment unit	canon.equipment_ node level WORK_UNIT + asset_profile	pump, motor, furnace	4-Hi tandem mill
L7 subunit	maint.maintainabl e_item level SUBUNIT	hydraulic unit, drive	mill hydraulic AGC
L8 maintainable item	maint.maintainabl e_item level MAINTAINABLE_I TEM	bearing, valve, motor	work-roll bearing
L9 part	maint.maintainabl e_item level PART	bearing SKU	specific bearing p/n

4. Criticality model (RCM)

asset_profile.criticality ∈ {A_CRITICAL, B_ESSENTIAL, C_GENERAL, D_LOW}.
Criticality drives: default PM intensity, work-order priority defaults, spares stocking depth (spec 05), and (later) which assets are first candidates for predictive (spec 08). Criticality **MAY** be set directly or derived from an FMECA roll-up (max RPN of the asset's failure modes).

5. Failure taxonomy (ISO 14224 + FMECA)

failure_mode columns (see M2_schema.sql):

Field	ISO 14224 sense
mode_desc / mode_code	failure mode — the observed effect (“fails to start”, “external leak”)
mechanism	failure mechanism — fatigue, wear, corrosion, electrical breakdown
cause	failure cause — design/install/use/maintenance root
effect	failure effect/impact — line stop / derate / safety / none
severity, occurrence, detectability (1–10)	FMECA scores
rpn (generated)	risk priority = S×O×D
assigned_strategy	RCM decision: RUN_TO_FAILURE / TIME_BASED / METER_BASED / CONDITION_BASED / PREDICTIVE
detection_method	how it's caught (operator, PM inspection,

condition monitoring, breakdown, sensor)

RCM decision logic (build as a guided workflow, not automated): for each significant failure mode, assign a strategy by technical feasibility + economic justification → the strategy becomes the basis for a maintenance_plan (condition/time/meter) or an explicit run-to-failure flag (excluded from PM scheduling). This is a Reliability-Engineer task surfaced in the UI; the system records the decision and links plan→failure_mode.

6. Seed taxonomies (sector-neutral core + Hero Steels template)

Ship a **generic** failure-mode starter set keyed to asset archetypes (rotating / static / electrical / material-handling / hydraulic / utility), then a **sector template** that specializes it. Open decision: adopt M1's MECH/ELECT/UTILITY/POWER stoppage codes as the v1 failure-category layer (fast, consistent with M1) vs a fuller ISO 14224 mode set (richer, more effort). Recommended: **map M1 codes → ISO 14224 categories** so both hold.

Hero Steels seed (illustrative rows):

Asset archetype	Failure mode	Mechanism	Default strategy
Mill work-roll bearing	overheating / vibration	fatigue/wear	CONDITION_BASED (vibration)
Hydraulic AGC	pressure loss / external leak	seal wear	TIME_BASED + inspection
Annealing furnace fan	bearing failure	wear	METER_BASED (run-hours)
EOT crane brake	brake fade	wear	TIME_BASED
Air compressor	air leak / overheating	wear/fouling	CONDITION_BASED

7. Interfaces

Registry CRUD via REST (spec 07): /v1/maint/assets/{id}/profile, /v1/maint/items, /v1/maint/failure-modes. Bulk asset/equipment import arrives via Manifold (SAP PM functional location + equipment → hierarchy + asset_profile).

8. NFRs

Hierarchy reads cached in serving; maintainable_item self-join bounded (≤4 levels); failure-mode lookups indexed by (tenant_id, asset_id).

9. Acceptance criteria

- **MUST** model an asset to L9 and attach failure modes with the ISO 14224 quartet.
- **MUST** compute rpn and allow an assigned_strategy per failure mode.
- **MUST** load the Hero Steels seed template and a generic non-steel template from the same schema (D11).
- **SHOULD** derive asset criticality from FMECA roll-up when not set manually.

10. Build phasing

M2-0: asset_profile + maintainable_item + failure_mode + seed templates + registry CRUD. Precedes the PM engine (03) which needs criticality + strategy.

11. Risks

Taxonomy churn (effective-date / version seed sets); inconsistent criticality scoring across plants (provide a documented banding rubric); incomplete L7–L9 data from legacy CMMS (allow asset-level failure modes when item-level is absent).

03 · Preventive Maintenance Engine — Developer Specification

Component owner: M2 Backend · **Phase:** M2-1 · **Depends on:** 01 Data Model, 02 Registry, 04 Work Orders · **Consumed by:** 04, 06, Monitoring

1. Purpose & scope

The engine that makes M2 *preventive*: it evaluates **triggers** and auto-generates **work orders** with their checklists before failure. **In scope:** trigger types, the due-scan algorithm, WO generation, compliance windows, floating re-anchoring, meter projection, basic condition rules. **Out of scope:** WO execution lifecycle (04); predictive forecasting (08).

2. Trigger model (MUST)

A maintenance_plan binds an asset (or asset_class for template plans) to one or more plan_trigger rows and a maintenance_task checklist. Trigger types (maint.trigger_type):

Type	Fires when	Driving data	Re-anchor on completion?
TIME_FIXED	fixed calendar interval since plan epoch	clock	no (fixed cadence)
TIME_FLOATING	interval elapsed since last completion	clock + WO history	yes
METER	meter crosses next threshold	meter_reading	yes (next = last + interval)
CONDITION	measured tag/meter breaches limit	sensor/meter	n/a (event-driven)
EVENT	a defined canonical	Event Spine	n/a

	event occurs (e.g. N breakdowns)		
INSPECTION_RO	scheduled rounds /	clock	yes
UTE	lubrication routes		

3. Due-scan algorithm (the core job)

Runs on **Airflow** at the per-tenant cadence (D2, default minutes; edge-deployable per D1). Idempotent — generating the same due WO twice is prevented by (plan_id, trigger_id, due_ts) uniqueness on open WOs.

```
for each active plan_trigger (tenant-scoped):
  case TIME_FIXED:
    due = epoch + ceil((now - epoch)/interval)*interval
    if now >= due and not already_generated(trigger, due): generate_wo(due)
  case TIME_FLOATING / INSPECTION_ROUTE:
    due = last_completed_ts(plan) + interval    (or plan.created if none)
    if now >= due and no open WO for trigger: generate_wo(due)
  case METER:
    latest = meter.current_value
    if latest >= trigger.next_due_meter and no open WO:
      generate_wo(due_ts=now); trigger.next_due_meter += meter_interval
    else: project_due_date(meter trend)  # for "due in ~X days" forecast
  case CONDITION:
    evaluate(condition_tag, condition_op, condition_value) against latest reading
    if breached: raise condition_alert; if alert.action=WO: generate_wo()
  case EVENT:
    on subscribed canonical event matching rule: generate_wo()
update trigger.next_due_ts / next_due_meter (scheduler cache)
```

generate_wo(...) creates a work_order (wo_type='PREVENTIVE' or 'CONDITION'/'INSPECTION', wo_source='PLAN'), copies maintenance_task → wo_task, reserves task_part spares (spec 05), sets due_ts = due + compliance_window_days, assigns per routing, and emits maintenance.pm.generated.

4. Meter handling

Meter readings (maint.meter_reading, TimescaleDB) arrive from PLC tags (automatic) or manual operator entry (open decision — support both). The engine maintains meter.current_value (cached cumulative) and computes delta_value for usage-rate, enabling **meter projection**: estimate the date a METER trigger will fire from recent average usage → surfaces “due in ~X days/hours” in Monitoring even though the trigger is usage-based.

5. Compliance window

Each generated PM has due_ts = ideal date + compliance_window_days. Completion within the window counts toward **PM compliance** (spec 06). Missed PMs are flagged (state

Skipped/overdue) and visible on the Monitoring board. Floating and route plans re-anchor next_due_ts from completed_ts, so cadence never drifts.

6. Scheduling vs idle time

Where shift/calendar data exists, the engine prefers slotting PM into planned/idle windows (maintenance_plan.requires_shutdown influences this) so preventive work does **not** create OEE Availability loss (the M2→M4 distinction, spec 06).

7. Interfaces (spec 07 detail)

POST /v1/maint/plans, POST /v1/maint/plans/{id}/triggers, POST /v1/maint/meters/{id}/readings, GET /v1/maint/plans/due?as_of=.... Emits maintenance.pm.generated, maintenance.pm.due, maintenance.condition.alert.

8. NFRs

- Due-scan **MUST** complete within one cadence interval for ≥100k active triggers/tenant (batch + index on plan_trigger.next_due_ts).
- **Idempotent**: re-runs never double-generate (uniqueness guard).
- **Edge-deployable** (D1): scan runs on-prem when the plant is disconnected; syncs generated WOs on reconnect.

9. Acceptance criteria

- **MUST** generate a WO with the correct checklist when any trigger type fires, exactly once.
- **MUST** re-anchor floating/route plans from completed_ts.
- **MUST** advance next_due_meter and project a due date from meter trend.
- **MUST** raise a condition_alert (and optionally a WO) on threshold breach.
- **SHOULD** prefer idle-time slotting when shift/calendar is available.

10. Build phasing

M2-1: time (fixed/floating) + meter triggers + due-scan + WO generation + compliance window; basic condition alerts. EVENT triggers and idle-time optimization are fast-follow.

11. Risks

PM “over-generation” / alarm fatigue (tunable thresholds, dedupe open WOs); clock vs meter drift (projection + reconciliation); manual meter entry gaps (fallback to time-based; flag stale meters).

04 · Work-Order Lifecycle & Maintenance Events — Developer Specification

Component owner: M2 Backend · **Phase:** M2-1 · **Depends on:** 01 Data Model, 02 Registry, 03 PM Engine · **Consumed by:** 05, 06, 07, Monitoring

1. Purpose & scope

The execution core: the **notification** → **work-order** lifecycle that all maintenance work flows through, plus how failures and M1 stoppages enter it. **In scope:** state machine, status transitions, corrective vs preventive paths, downtime capture, labour/parts capture, M1 bootstrap, offline capture, events. **Out of scope:** spares stock mechanics (05), KPI math (06).

2. Key concepts (MUST)

- Adopt the **notification** – **order** split (SAP PM / Maximo pattern): a notification is *a report that something needs attention*; a work_order is *the authorised, planned, costed job*. This keeps demand signal (and its KPIs) separate from execution.
- A breakdown is recorded **once** as a canon.event downtime row; the work_order and failure_event both reference it via downtime_event_ref. M4 aggregates that same row. **Never double-count.**

3. State machine

stateDiagram-v2

```
[*] --> Reported: breakdown / defect / PM trigger / condition alert
Reported --> Notification: log problem (asset, failure mode, symptom)
Notification --> Planned: scope tasks, parts, labour, permits
Planned --> Scheduled: slot into calendar
Scheduled --> Assigned: crew + parts reserved
Assigned --> InProgress: work starts (downtime clock if corrective)
InProgress --> Completed: log labour, parts, readings, root cause
Completed --> Closed: verify + capture cost
Notification --> RejectedDuplicate: merged/duplicate
Scheduled --> Cancelled: no longer needed
Closed --> [*]
```

work_order.status enum: REPORTED, PLANNED, SCHEDULED, ASSIGNED, IN_PROGRESS, COMPLETED, CLOSED, CANCELLED, REJECTED_DUPLICATE.
Transitions are validated server-side; each transition is audit-logged (who/when) and emits an event.

4. Two entry paths

Path	Trigger	What's created	Downtime?
Corrective / breakdown	failure occurs (operator/sensor/M	notification → work_order	yes — Availability loss

	1 stoppage)	(CORRECTIVE) + failure_event + canon.event downtime row; actual_start..actual_end ⇒ repair time (MTTR)	
Preventive	plan_trigger fires (spec 03)	work_order (PREVENTIVE) with checklist pre- attached	ideally none (idle- time)

5. M1 stoppage bootstrap (the cheapest on-ramp)

Where M1 is live, breakdown signal already exists. Subscribe to M1 downtime.logged events; for stoppages with category MECH/ELECT/UTILITY and duration ≥ threshold:

```
on downtime.logged(category in {MECH,ELECT,UTILITY}, duration>=T):
  upsert failure_event(asset, down_ts, up_ts, source=M1_STOPPAGE,
    downtime_event_ref = that.canon.event.id)
  optionally open work_order(CORRECTIVE, wo_source=M1_STOPPAGE)
```

This lights up MTBF/MTTR analytics **before** any CMMS is connected. Mapping of M1 codes → failure categories is configurable (spec 02 §6).

6. Capture (first-party, no CMMS)

When the client has no CMMS, M2 provides capture UIs reusing M1 patterns: **offline-tolerant PWA** (queue locally, sync on reconnect), large touch targets for shop-floor use, two-state DRAFT→SUBMITTED then supervisor APPROVE→CLOSE, post-close edits via audited change request. Sub-forms: labour, parts, readings, root cause.

7. Cost capture at close

On Completed→Closed: resolve wo_labor.cost (hours × canon.cost_rate **effective at the work date** — D7), wo_part.cost (qty × unit_cost), and downtime_cost (downtime_h × downtime_rate). Roll up to work_order.labor_cost/part_cost/downtime_cost (cached; authoritative lines retained). Detail in spec 05/06.

8. Events (spec 07)

Publishes: maintenance.notification.created, maintenance.workorder.created|assigned|started|completed|closed, maintenance.failure.logged. Consumes: downtime.logged (M1/M4), maintenance.condition.alert (03), sensor.reading (meters).

9. NFRs

- Capture path **offline-tolerant** and never blocks (D1); sync is idempotent (client-generated dedupe key).
- All transitions audited; status changes emit at-least-once events with idempotent consumers.

10. Acceptance criteria

- **MUST** enforce valid status transitions and audit each.
- **MUST** record a corrective WO's repair window and link the single shared downtime_event_ref.
- **MUST** bootstrap failure_event from qualifying M1 stoppages without operating a CMMS.
- **MUST** capture labour, parts, and root cause before Closed.
- **SHOULD** support offline capture with conflict-free sync.

11. Risks

Double-counting downtime (mitigate: single downtime_event_ref, enforced); notification spam (dedupe/merge to REJECTED_DUPLICATE); offline sync conflicts (client dedupe keys + last-writer rules); incomplete root-cause capture (require failure_mode on close for breakdowns).

05 • Spares / MRO & Maintenance Cost — Developer Specification

Component owner: M2 Backend · **Phase:** M2-2 · **Depends on:** 04 Work Orders, platform 07 Financial Impact · **Consumed by:** 06 KPIs, Reporting

1. Purpose & scope

A proportionate spares/MRO model and the maintenance cost engine. Enough to drive availability and cost analytics — **not** a full procurement suite. **In scope:** spare catalogue, equipment BOM, reservation/consumption against WOs, stock ledger, reorder, criticality stocking, cost model + Financial Impact integration. **Out of scope (EAM):** purchase orders, supplier management, depreciation.

2. Key concepts (MUST)

- Spares (spare_part) are keyed to **maintainable items** via spare_bom (which spares serve which item/asset) — answers “what does this asset need” and “stockout risk for a critical asset”.

- Stock is moved only through spare_txn (RECEIVE/ISSUE/RESERVE/UNRESERVE/ADJUST/RETURN); on_hand_qty/reserved_qty are derived caches kept consistent with the ledger.
- Cost is **never** estimated where a rate exists: resolve the canon.cost_rate **effective at the event date** (D7), not today's rate.

3. Spares lifecycle

flowchart LR

```

REQ["WO needs part (task_part / ad-hoc)"] --> RES["RESERVE (spare_txn) → reserved_qty++"]
RES --> ISS["ISSUE on use (spare_txn) → on_hand--; wo_part.cost booked"]
ISS --> CHK{"on_hand <= reorder_point?"}
CHK -- yes --> RO["Flag reorder (qty=reorder_qty) → Monitoring/Reporting"]
CHK -- no --> OK["ok"]

```

4. Criticality-based stocking (RCM)

Stock depth follows asset/failure criticality (spec 02): a spare serving an A_CRITICAL single-point-of-failure asset is stocked even if rarely used; D_LOW items can be order-on-demand. Surface a **stockout-risk** view: critical assets whose required spares are below reorder point.

5. Maintenance cost model

$\text{maintenance_cost}(\text{WO}) = \text{labour} + \text{parts} + \text{downtime}$

Component	Source	Rate
Labour	$\Sigma \text{wo_labor.hours}$	$\times \text{labour cost_rate}$ (effective-dated, by skill if available)
Parts	$\Sigma \text{wo_part.qty} \times \text{unit_cost}$	spare unit cost (or PO cost when present)
Downtime (corrective only)	$\text{failure_event.downtime_h}$	$\times \text{downtime cost_rate}$ from Financial Impact Layer

Roll-ups (materialized to serving / kpi_asset_daily): **cost-per-asset, cost-per-failure-mode, planned vs unplanned cost, MRO consumption value**. These are the numbers that justify a reliability programme and feed the executive financial report.

6. Financial Impact integration

M2 does not own rates — it consumes the platform **Financial Impact Layer** (platform spec 07): cost-rate registry with effective_from/to and “rate as of” provenance (D7). M2 passes the **event date** so pricing is historically correct. Downtime cost uses the same rate basis M4 uses for Availability-loss pricing, so maintenance cost and OEE-loss cost agree.

7. Interfaces (spec 07)

/v1/maint/spares (CRUD), /v1/maint/spares/{id}/txns,
/v1/maint/work-orders/{id}/parts, /v1/maint/spares/reorder (list below reorder point).
Emits maintenance.spare.issued, maintenance.spare.reorder.

8. NFRs

Stock mutations transactional (ledger + cache in one tx); concurrent issues serialized per spare_id; cost resolution deterministic and reproducible (store the resolved rate_ref + value on the line).

9. Acceptance criteria

- **MUST** keep on_hand_qty/reserved_qty consistent with the spare_txn ledger at all times.
- **MUST** book wo_part.cost and wo_labor.cost using the rate effective at the work date (D7).
- **MUST** flag reorder when on-hand $\leq$ reorder point and expose stockout risk for critical assets.
- **SHOULD** roll up cost-per-asset and cost-per-failure-mode to the serving layer.

10. Build phasing

M2-2: catalogue + BOM + txn ledger + WO consumption + cost rollups + Financial Impact wiring. Reorder/stockout views fast-follow. Procurement is out of scope (future EAM).

11. Risks

Stock/ledger drift (single-transaction writes + periodic reconciliation); missing unit costs (fall back to last-known / flag); rate ambiguity (always store resolved rate on the line for auditability).

06 • KPIs, Analytics & Financial Impact — Developer Specification

Component owner: M2 Backend + Analytics · **Phase:** M2-2 · **Depends on:** 04 Work Orders, 05 Cost, platform 04 UIL, 07 Financial Impact · **Consumed by:** UIL, Monitoring, Reporting

1. Purpose & scope

M2's contribution to the **shared KPI registry** (platform 04 UIL): the maintenance KPI family, their exact definitions and SQL, the rollup grain, and the **M2→M4 reconciliation**

rule. Every formula is the industry-standard definition; OEE-linked terms reuse the platform's ISO 22400 definitions — **no parallel truth**.

2. KPI definitions (MUST match these)

KPI	Definition	Notes
MTBF	$\text{operating_time} \div \text{failures}$	repairable assets
MTTR	$\Sigma \text{repair_time} \div \text{repairs}$	maintainability
MTTF	$\text{operating_time} \div \text{failures}$	non-repairable items
MTBM	$\text{operating_time} \div \text{all maintenance actions (PM+CM)}$	total demand
Inherent Availability	$\text{MTBF} \div (\text{MTBF} + \text{MTTR})$	reliability-based (distinct from OEE availability)
PM Compliance	$\text{completed_PM} \div \text{scheduled_PM} \times 100$	within compliance window; target $\geq 90\%$
Schedule Compliance	$\text{on_time_WOs} \div \text{scheduled_WOs} \times 100$	planning discipline
PMP (Planned Maint %)	$\text{planned_maint_hours} \div \text{total_maint_hours} \times 100$	proactive ratio
Planned:Unplanned	$\text{planned_WOs} \div \text{unplanned_WOs}$	maturity
Backlog	$\text{open ready-to-schedule WO hours} \div \text{weekly capacity}$	weeks of work
Failure Pareto / bad actors	failures (or downtime/cost) grouped by asset & failure_mode	RCM focus

3. Reference SQL (illustrative; computed into kpi_asset_daily / mart)

-- MTBF, MTTR, inherent availability per asset over a window

```

WITH f AS (
  SELECT asset_id,
         COUNT(*) AS failures,
         SUM(downtime_h) AS downtime_h,
         SUM(EXTRACT(EPOCH FROM (actual_end-actual_start)))/3600.0 AS
repair_h
  FROM maint.failure_event fe
 LEFT JOIN maint.work_order wo ON wo.failure_event_id = fe.failure_event_id
 WHERE fe.tenant_id = :t AND fe.down_ts >= :from AND fe.down_ts < :to
 GROUP BY asset_id
)
SELECT asset_id,
       (:operating_h / NULLIF(failures,0)) AS mtbf_h,
       (repair_h / NULLIF(failures,0)) AS mtrr_h,
       (:operating_h / NULLIF(failures,0)) /

```

```

        NULLIF((:operating_h/NULLIF(failures,0)) +
        (repair_h/NULLIF(failures,0)),0)    AS availability --
MTBF/(MTBF+MTTR)
FROM f;

-- PM compliance per asset
SELECT asset_id,
        100.0 * COUNT(*) FILTER (WHERE status='CLOSED' AND completed_ts <=
due_ts)
        / NULLIF(COUNT(*) FILTER (WHERE wo_type='PREVENTIVE'),0) AS
pm_compliance
FROM maint.work_order
WHERE tenant_id=:t AND created_ts >= :from AND created_ts < :to
GROUP BY asset_id;

```

operating_time comes from shift/calendar planned production time minus downtime (consistent with ISO 22400) — sourced from the canonical serving layer, not recomputed differently here.

4. Rollup grain & publication

- Materialize daily to maint.kpi_asset_daily; aggregate up the ISA-95 hierarchy (asset→work_center→site) and by failure_mode for Pareto.
- **Publish** each KPI to the shared **KPI registry** (UIL) with: kpi_code, scope (asset/site/tenant), period, value, unit, definition_ref. UIL handles cross-module synthesis (e.g. maintenance→quality→yield); M2 just contributes its metrics.

5. M2 ↔ M4 reconciliation (MUST)

A breakdown is one canon.event downtime row (downtime_event_ref). **M2** derives failure-centric metrics (MTBF, MTTR, inherent availability); **M4** derives time-based **OEE availability** (Operating ÷ Planned Production Time). The two availability figures are *different metrics by design*, but both consume the **same downtime hours**, so they reconcile and never contradict. Acceptance test: $\sum$ M2 breakdown-downtime_h for an asset/period == $\sum$ M4 Availability-loss hours attributed to breakdowns for the same asset/period.

6. Financial Impact tagging

Each KPI is priced where a cost_rate exists (spec 05, D7): downtime cost = downtime_h × downtime rate; repair cost = labour+parts; **avoided-failure value** (future, when a condition catch prevents a breakdown). Priced KPIs flow to the executive financial report and to alerts (“£X/yr avoided”). Pricing uses the rate effective at the event date so historical reports stay correct.

7. Interfaces (spec 07)

GET /v1/maint/kpis?scope=&asset=&from=&to=, GET /v1/maint/analytics/pareto?dim=failure_mode. Emits maintenance.kpi.published. Live tiles (due/overdue PM, open breakdowns, asset health) pushed to Redis for Monitoring.

8. NFRs

KPI rollups run on Airflow at tenant cadence; mart queries sub-second for dashboards (ClickHouse/columnar); definitions versioned (definition_ref) so a number is always traceable to its formula.

9. Acceptance criteria

- **MUST** compute MTBF/MTTR/inherent-availability/PM-compliance per the \$2 definitions.
- **MUST** reconcile breakdown downtime hours with M4 (\$5 test).
- **MUST** publish KPIs to the shared registry with definition provenance.
- **MUST** price downtime/repair via the effective-dated rate (D7).
- **SHOULD** expose failure-mode Pareto and bad-actor ranking.

10. Build phasing

M2-2: core KPIs + daily rollup + registry publish + financial tagging + M4 reconciliation test. Backlog/wrench-time and avoided-failure value fast-follow.

11. Risks

Definition drift across modules (single registry + versioned definitions); operating-time source ambiguity (take from canonical serving, document it); small-sample MTBF noise (show confidence/minimum-failures guard).

07 • APIs, Events & Integration — Developer Specification

Component owner: M2 Backend + Platform · **Phase:** M2-1→2 · **Depends on:** platform 03 Manifold, 08 Platform/Security · **Consumed by:** all M2 UIs, UIL, Monitoring, Reporting, M4

1. Purpose & scope

M2's external contracts: REST APIs, domain events, the **Manifold connector mapping** for ingesting an existing CMMS/EAM (SAP PM first), and the module boundaries. **In scope:** endpoint catalogue, event catalogue, SAP PM—canonical field map, boundary contracts. **Out of scope:** internal service design (specs 03–06).

2. API conventions (inherited — MUST)

REST, versioned /v1/..., OpenAPI-documented, **OIDC** (Keycloak) secured, **idempotent** writes (client idempotency key), cursor pagination, RFC-7807 error bodies, all requests tenant-scoped. (Platform 00 \$5.)

3. REST endpoint catalogue (v1)

Area	Method & path	Purpose
Registry	GET/POST /v1/maint/assets/{id}/profile · GET/POST /v1/maint/items · GET/POST /v1/maint/failure-modes	reliability registry CRUD
Plans	GET/POST /v1/maint/plans · POST /v1/maint/plans/{id}/triggers · POST /v1/maint/plans/{id}/tasks · GET /v1/maint/plans/due?as_of=	PM plans, triggers, checklists, due list
Meters	POST /v1/maint/meters · POST /v1/maint/meters/{id}/readings · GET /v1/maint/meters/{id}	usage counters
Notifications	POST /v1/maint/notifications · POST /v1/maint/notifications/{id}/convert	problem reports → WO
Work orders	GET/POST /v1/maint/work-orders · PATCH /v1/maint/work-orders/{id}/status · POST /.../labor · POST /.../parts · POST /.../tasks	lifecycle + capture
Failures	GET/POST /v1/maint/failures	failure events + analytics
Spares	GET/POST /v1/maint/spares · POST /v1/maint/spares/{id}/txns · GET /v1/maint/spares/reorder	MRO
KPIs/analytics	GET /v1/maint/kpis · GET /v1/maint/analytics/pareto	metrics + Pareto
Health (future)	GET /v1/maint/assets/{id}/hea	RUL/health (spec 08, empty

lth in v1)

Writes that produce canonical facts call the **canonical write-back API** (platform 01 §8) so canon.event rows + specializations are created consistently with lineage.

4. Domain events (Kafka/Redpanda; at-least-once, idempotent consumers)

Direction	Topic / event	Payload (key fields)
Publish	maintenance.notification.created	notif_id, asset, failure_mode, severity
Publish	maintenance.workorder.created\\assigned\\started\\completed\\closed	wo_id, asset, type, status, costs
Publish	maintenance.failure.logged	failure_event_id, asset, mode, downtime_h, downtime_event_ref
Publish	maintenance.pm.due / maintenance.pm.generated	plan_id, trigger_id, asset, due_ts
Publish	maintenance.condition.alert	asset, tag, value, threshold, severity
Publish	maintenance.spare.issued / maintenance.spare.reorder	spare_id, qty, on_hand
Publish	maintenance.kpi.published	kpi_code, scope, period, value
Consume	downtime.logged (M1/M4)	→ bootstrap failure_event (spec 04 §5)
Consume	sensor.reading / meter feed	→ meter triggers + condition rules
Consume	mapping.confirmed (Manifold)	→ CMMS field maps activated

5. Manifold connector mapping — SAP PM (primary CMMS target)

M2 does not build connectors; **Manifold** (platform 03) ingests and maps. M2 supplies the **maintenance sector template** (canonical target fields). SAP PM → canonical map:

SAP PM object

Functional Location

Equipment

Equipment BOM

Task List

Maintenance Plan (MPLA)

Notification

Work Order (order header/ops)

Measurement Point/Document

Material/Spare

Maximo/Infor follow the same target via their own maps. Connector priority (open decision): **SAP PM first** (Hero Steels runs SAP) per platform D4 sequence (files/DB first, then SAP/API). Dedup & source-key resolution handled by Manifold (no duplicate assets when capture + CMMS both present).

6. Module boundaries (contracts)

With	M2 provides	M2 consumes	Shared
M1	enriches stoppages → failures	downtime.logged, capture patterns	canon.event
M4 (OEE)	failure classification, MTTR	—	the single downtime_event_ref (never double-count)
UIL	maintenance KPIs	cross-module synthesis	KPI registry
Monitoring	asset-health/due-PM tiles (Redis)	live cache infra	streaming path
Financial Impact	what to price	cost-rate engine (D7)	cost_rate
Reporting	reliability/cost report data	render/schedule	same KPI numbers

7. NFRs

APIs stateless + horizontally scalable; events idempotent; connector sync incremental (CDC where SAP allows) and edge-tolerant (D1); contracts versioned so consumers don't break on internal change.

8. Acceptance criteria

- **MUST** expose the v1 endpoints with OpenAPI specs and OIDC enforcement.
- **MUST** publish/consume the listed events with idempotent handling.
- **MUST** ingest a sample SAP PM export via Manifold into maint.* using the \$5 map.
- **MUST** route all canonical facts through the write-back API (lineage preserved).
- **SHOULD** support incremental CMMS sync, not just full reload.

9. Build phasing

M2-1: internal REST + events + M1-stoppage consume. **M2-2:** KPI/analytics endpoints. **M2-3:** Manifold SAP PM connector + sector template + dedup.

10. Risks

Connector variance across SAP versions (template + Manifold mapping review queue, D5); duplicate assets from capture+CMMS (Manifold dedup/source-key map); event ordering (idempotent consumers + event timestamps).

08 · Predictive (PdM) — Future-Scope Interfaces — Developer Specification

Component owner: M2 + ML Platform (later) · **Phase:** M2-4 (GATED — do not build in v1) · **Depends on:** 01 Data Model, 04 Work Orders, 06 KPIs · **Status:** interfaces & data hooks only

1. Purpose & scope

Predictive maintenance is **future scope** (platform D9: rule-based now, ML later; operating-model post-≈Nov 2026, gated on accumulated data). This spec defines the **forward-compatible interfaces and data hooks v1 must honour** so PdM is a later *switch-on*, not a re-model. **It does NOT specify models to build now.** Build emphasis in v1 = preventive (specs 02–07).

2. What v1 MUST do (so PdM is unblocked later)

- **Capture sensor_reading on the canonical time-series spine** from day one where any sensor/meter exists (even sparse data has future value). Vibration, temperature, current, pressure, flow, acoustic.
- **Label failure_event** with confirmed failure_mode (mechanism/cause) — these are the **training labels**. Enforced on breakdown close (spec 04).
- **Keep operating context joinable** to readings (load, product, shift) via the Event Spine.
- **Expose the ML-readiness (History) gate** in the readiness model (platform 01 §16) so the UI honestly reports which assets have enough history to train — no v1 predictive *promises*, only predictive *readiness*.
- **Leave the output slot ready:** maint.asset_health (health grade, anomaly_score, rul_hours, model_ref, advisory) exists with **no producer** in v1.

3. Target architecture — ISO 13374 / MIMOSA OSA-CBM (six blocks)

When built, PdM implements the standard open pipeline as services:

flowchart LR

DA["1 Data Acquisition"] --> DM["2 Data Manipulation (features, FFT)"]

DM --> SD["3 State Detection (anomaly)"]

SD --> HA["4 Health Assessment"]

HA --> PA["5 Prognostic Assessment (RUL)"]

PA --> AG["6 Advisory Generation"]

AG --> WO["Condition/Predictive WO + alert"]

AG --> HEALTH["maint.asset_health (output slot)"]

Each block is a service contract (stub interfaces defined now, implementations later).

Feature store = **Feast**; model registry/lifecycle = **MLflow** (reserved per platform stack \$4).

4. Interface stubs (defined now, return “not available” in v1)

Interface	v1 behaviour	Future behaviour
GET /v1/maint/assets/{id}/health	200 with status:"not_available", readiness:{history_pct}	health grade + anomaly score
GET /v1/maint/assets/{id}/rul	not_available + readiness	RUL hours + confidence
POST /v1/maint/condition/evaluate	basic threshold rule (spec 03)	model-based state detection
event maintenance.health.assessed	not emitted	emitted by OSA-CBM service
maint.asset_health table	empty (no producer)	populated by prognostics service

5. ML approach (when gated open)

Phased by data availability: **(1) anomaly detection** (unsupervised — needs no failure labels; earliest win); **(2) RUL regression / survival models** (need run-to-failure histories); **(3) failure-mode classification** (need labelled failure_event). Ladder follows the platform’s descriptive→diagnostic→**predictive**→prescriptive progression; v1 sits at descriptive+diagnostic.

6. The data gate (open decision)

Define the threshold that flips an asset to “predictive-ready”, e.g. ≥N months of sensor history **and** ≥K labelled failures of a mode. Until then the readiness API reports history_pct < gate. This decision is captured for the user (open items, spec 00 \$8).

7. Techniques supported (when built)

vibration (FFT/envelope), thermography (trending), oil/tribology (particle/contaminant), motor-current signature (MCSA), ultrasound/acoustic — each maps a sensor_reading stream to a diagnostic; the **P-F curve** lead time (P→F interval) is what the advisory monetizes (failure avoided, spec 06 §6).

8. Explicit “do NOT build in v1”

No ML training pipelines, no Feast/MLflow deployment, no prognostics service, no asset_health producer, no predictive promises in UI. **Only** the data capture, labelling, readiness gate, and stub interfaces above.

9. Acceptance criteria (v1)

- **MUST** persist sensor_reading and labelled failure_event so a future model can train on history.
- **MUST** expose health/rul stub endpoints returning readiness, not fabricated values.
- **MUST** keep maint.asset_health and maintenance.health.assessed defined but unproduced.
- **SHOULD** report per-asset history/readiness % toward the predictive gate.

10. Build phasing

M2-4 (gated): only after data accumulation + a go decision. Anomaly detection first, then RUL, then classification. Until then this spec is a contract, not a workstream.

11. Risks

Premature ML (mitigate: hard data gate); unlabelled failures (enforce failure_mode on close); sensor data gaps (capture whatever exists; readiness reflects reality); over-promising predictive to clients (UI shows readiness, never fake RUL).

09 • Security, RBAC, NFRs & Acceptance — Developer Specification

Component owner: M2 + Platform/Security · **Phase:** cross-cutting (M2-0 onward) ·
Depends on: platform 08 Platform & Security · **Consumed by:** all M2 specs

1. Purpose & scope

The cross-cutting requirements every M2 service must meet: roles & permissions, multi-tenancy, non-functionals, the consolidated acceptance criteria, and the test plan. M2 **inherits** the platform security model (platform 08 — OIDC/Keycloak, TLS, encryption at

rest, tenant isolation, audit, OpenTelemetry) and adds maintenance-specific roles and targets.

2. RBAC (extends the M1 role model)

M1 roles: Operator / Supervisor / Plant-Head / Admin. M2 adds maintenance personas (mapped to Keycloak groups; row-level scoping by site/area as in M1).

Role	Can	Cannot
Technician	execute assigned WOs, log labour/parts/readings/root cause, raise notifications	edit plans, change cost rates, close others' WOs
Maintenance Planner	create/edit plans & triggers, schedule & assign WOs, manage backlog, manage spares	change RBAC, edit cost rates
Reliability Engineer	manage failure modes/FMECA/criticality, condition rules, (future) predictive config	execute WOs outside scope
Maintenance Manager	read all KPIs/cost/analytics, approve high-cost WOs	edit operational records
Admin	masters, integration (connectors), workflow config, role assignment	—
<i>(inherited)</i> Plant-Head	read-only analytics across lines	edits

Cost-rate management belongs to the Financial Impact Layer's owner (platform 07), not M2 roles (D7 governance).

3. Multi-tenancy & data protection (MUST)

- `tenant_id` on every maint.* row; all queries tenant-scoped at the data-access layer (D8); no cross-tenant path exists.
- Logical isolation default; physical on client request (D1/D8).
- Encryption in transit (TLS) + at rest; per-tenant keys per platform policy.
- **Audit:** every WO create/transition/close, cost booking, plan change, mapping, and export logged (who/what/when) — reuse the M1 audit pattern; post-close edits require an audited change request.

4. Non-functional requirements

Concern	Target
Latency	PM due-scan + condition eval within tenant cadence (default minutes, D2); capture

	never blocks
Scale	≥100k active triggers and ≥10M failure/WO rows per tenant; meter readings at TimescaleDB scale
Availability	99.5% for serving/monitoring; capture & PM scan edge-deployable and offline-tolerant (D1)
Offline	first-party capture queues locally, syncs idempotently on reconnect
Integrity	idempotent pipelines; one downtime row shared with M4; stock ledger = source of truth for on-hand
Observability	OTel traces, Prometheus metrics, structured logs — tagged by tenant + service
Reproducibility	costed lines store the resolved rate (audit); KPIs store definition_ref

5. Consolidated acceptance criteria (module-level)

- **MUST** stand up maint.* from M2_schema.sql against canon with zero errors.
- **MUST** run the full notification→WO lifecycle (corrective + preventive) with valid, audited transitions.
- **MUST** generate PMs from time/meter/condition triggers exactly once, honouring the compliance window.
- **MUST** bootstrap failures from M1 stoppages with no CMMS present.
- **MUST** compute the maintenance KPI family per spec 06 and reconcile breakdown downtime with M4.
- **MUST** price labour/parts/downtime via effective-dated rates (D7).
- **MUST** keep predictive as readiness-only (no fabricated RUL) per spec 08.
- **MUST** enforce RBAC + tenant isolation + audit on every write.
- **SHOULD** ingest a sample SAP PM export via Manifold; **SHOULD** support offline capture; **SHOULD** load both a steel and a non-steel seed (D11).

6. Test plan

Layer	Tests
Unit	trigger date math (fixed/floating/meter projection), KPI formulas, cost resolution, state-transition guards
Integration	due-scan → WO generation → close → KPI rollout; M1 stoppage → failure_event; spare reserve→issue→ledger
Contract	OpenAPI conformance; event schema

Data quality	compatibility; canonical write-back lineage failure has ISO 14224 quartet; one downtime_event_ref per breakdown; on_hand == ledger
Reconciliation	M2 breakdown downtime_h == M4 Availability-loss hours (spec 06 \$5)
Non-functional	due-scan throughput at 100k triggers; offline capture sync; tenant-isolation negative tests
Acceptance	the \$5 MUSTs, scripted against the Hero Steels seed + a non-steel seed

7. Build phasing

Security/RBAC/audit land in **M2-0** (before any capture). NFR targets are verified continuously; the reconciliation and offline tests gate **M2-2** and **M2-1** respectively.

8. Risks

Privilege creep (least-privilege default, periodic review); audit gaps on offline edits (change-request workflow); tenant-isolation regressions (negative tests in CI); NFR drift under real meter volume (load test early with TimescaleDB).

10 • Glossary — M2 Maintenance Intelligence

Terms specific to M2. Platform-wide terms (canonical model, Event Spine, Manifold, readiness, tenant) are in the platform glossary ../Dev_Handover/09_Glossary.md.

Maintenance strategies (EN 13306)

- **Corrective maintenance** — work after fault recognition to restore function. *Immediate* (breakdown/emergency) or *deferred* (planned corrective).
- **Preventive maintenance** — work before failure. Splits into *predetermined* and *condition-based*.
- **Predetermined maintenance** — scheduled at a fixed time/usage interval, no condition check.
- **Condition-based maintenance (CBM)** — act when a measured parameter crosses a limit.
- **Predictive maintenance (PdM)** — CBM guided by a forecast of degradation (RUL). *Future scope*.
- **Run-to-failure (RTF)** — a deliberate RCM choice to not PM a low-criticality item.

Reliability data (ISO 14224)

- **Failure mode** — the effect by which a failure is observed (e.g. “fails to start”).
- **Failure mechanism** — the physical/chemical process (fatigue, wear, corrosion).
- **Failure cause** — the root circumstance (design/install/use/maintenance).
- **Failure effect/impact** — consequence on function/plant.
- **Maintainable item** — the lowest level routinely repaired/replaced (ISO 14224 L8).
- **Equipment taxonomy (L1-L9)** — industry→plant→...→equipment
unit→subunit→item→part.

Reliability methods

- **RCM** — Reliability-Centered Maintenance: assign a strategy per failure mode by feasibility + economics.
- **FMECA** — Failure Mode, Effects & Criticality Analysis; **RPN** = severity × occurrence × detectability.
- **Criticality** — A_CRITICAL / B_ESSENTIAL / C_GENERAL / D_LOW; drives PM intensity, WO priority, spares depth.

KPIs

- **MTBF** — mean time between failures (operating time ÷ failures).
- **MTTR** — mean time to repair (Σ repair time ÷ repairs).
- **MTTF / MTBM** — mean time to failure (non-repairable) / between maintenance (all actions).
- **Inherent availability** — $MTBF \div (MTBF + MTTR)$; reliability-based (distinct from time-based OEE availability).
- **PM compliance** — completed ÷ scheduled PMs (within window) × 100; target ≥90%.
- **Schedule compliance, PMP** (planned maintenance %), **planned:unplanned, backlog, failure Pareto / bad actors**.

Predictive (future)

- **OSA-CBM / ISO 13374** — open architecture, six functional blocks: data acquisition → manipulation → state detection → health assessment → prognostics → advisory.
- **P-F curve** — from potential failure (P, first detectable) to functional failure (F); the **P-F interval** is the lead time PdM buys.
- **RUL** — remaining useful life (time-to-failure forecast).
- **Anomaly detection / survival model / failure classification** — the ML approaches, phased by data availability.

M2 objects (schema)

- **asset_profile** — reliability attributes on a canon WORK_UNIT (criticality, nameplate, default strategy).
- **maintainable_item** — ISO 14224 L7-L9 subtree under an asset.
- **maintenance_plan / plan_trigger / maintenance_task** — PM definition: what, when, checklist.

- **notification / work_order** — problem report → authorised job (SAP-style split).
 - **failure_event** — a failure occurrence; links the shared canonical downtime_event_ref.
 - **meter / meter_reading** — usage counters for meter-based PM.
 - **spare_part / spare_bom / wo_part / spare_txn** — MRO catalogue, asset BOM, consumption, stock ledger.
 - **condition_alert / asset_health** — basic CBM alert (v1) / future RUL-health output slot (no v1 producer).
-

Appendix A • M2_schema.sql (runnable PostgreSQL DDL)

```
--
=====
-- Zedra! · M2 Maintenance Intelligence — Reference Schema (PostgreSQL 15+)
-- Status: v1 for build (preventive scope). Predictive hooks present, build deferred.
-- Conventions: follows canonical DDL spec (02_CanonicalDataModel_Spec §5):
-- lower_snake_case · surrogate BIGINT ids · UUID tenant_id · TIMESTAMPTZ
(UTC)
-- JSONB `ext` extension namespace · FK to canon.* · units embedded in column
name.
-- Assumes the canonical core already exists:
-- canon.equipment_node(node_id) -- ISA-95 hierarchy incl. WORK_UNIT (=
asset)
-- canon.event(event_id)          -- the Event Spine (downtime_event lives here)
-- canon.cost_rate(rate_id ...)   -- effective-dated rates (D7) [referenced by text
rate code]
-- canon.personnel(person_id)
-- Time-series tables (meter_reading) are TimescaleDB hypertable candidates.
--
=====

CREATE SCHEMA IF NOT EXISTS maint;
SET search_path = maint, canon, public;

-- -----
-- 0. Enumerated types
-- -----
CREATE TYPE maint.criticality AS ENUM
('A_CRITICAL','B_ESSENTIAL','C_GENERAL','D_LOW');
CREATE TYPE maint.maint_strategy AS ENUM
('RUN_TO_FAILURE','TIME_BASED','METER_BASED','CONDITION_BASED','PRED
ICTIVE');
CREATE TYPE maint.item_level AS ENUM
('SUBUNIT','MAINTAINABLE_ITEM','PART'); -- ISO 14224 L7-L9
CREATE TYPE maint.trigger_type AS ENUM
```

```

('TIME_FIXED','TIME_FLOATING','METER','CONDITION','EVENT','INSPECTION_
ROUTE');
CREATE TYPE maint.wo_type          AS ENUM
('CORRECTIVE','PREVENTIVE','CONDITION','INSPECTION');
CREATE TYPE maint.wo_source        AS ENUM
('PLAN','NOTIFICATION','BREAKDOWN','CONDITION_ALERT','M1_STOPPAGE','C
MMS_IMPORT');
CREATE TYPE maint.wo_status        AS ENUM
('REPORTED','PLANNED','SCHEDULED','ASSIGNED','IN_PROGRESS','COMPLET
ED','CLOSED','CANCELLED','REJECTED_DUPLICATE');
CREATE TYPE maint.notif_status     AS ENUM
('OPEN','PLANNED','CONVERTED','REJECTED');
CREATE TYPE maint.task_type       AS ENUM
('INSPECT','REPLACE','LUBRICATE','CALIBRATE','CLEAN','ADJUST','TEST');
CREATE TYPE maint.task_result     AS ENUM ('PASS','FAIL','NA');
CREATE TYPE maint.meter_type      AS ENUM
('RUNTIME_HOURS','CYCLES','TONNAGE','DISTANCE_KM','STARTS','UNITS_PR
ODUCED');
CREATE TYPE maint.spare_txn_type   AS ENUM
('RECEIVE','ISSUE','RESERVE','UNRESERVE','ADJUST','RETURN');
CREATE TYPE maint.detection_method AS ENUM
('OPERATOR','PM_INSPECTION','CONDITION_MONITORING','BREAKDOWN','SE
NSOR_ALERT');

```

```

-----
-- 1. Asset reliability profile (1:1 extension of a canon WORK_UNIT / asset)
-----

```

```

CREATE TABLE maint.asset_profile (
  asset_id      BIGINT PRIMARY KEY REFERENCES
  canon.equipment_node(node_id),
  tenant_id     UUID NOT NULL,
  criticality   maint.criticality NOT NULL DEFAULT 'C_GENERAL',
  default_strategy maint.maint_strategy NOT NULL DEFAULT 'TIME_BASED',
  manufacturer  TEXT,
  model         TEXT,
  serial_no     TEXT,
  commission_date DATE,
  design_life_y NUMERIC,           -- design life, years
  acquisition_value NUMERIC,      -- hook for later EAM (depreciation) — not
used in v1
  iso14224_class TEXT,             -- e.g. 'PU' pump, 'MO' motor, 'HE' heat-
exchanger
  source_ref    TEXT,
  ext           JSONB NOT NULL DEFAULT '{}':jsonb,
  UNIQUE (tenant_id, asset_id)
);
COMMENT ON TABLE maint.asset_profile IS 'Maintenance/reliability attributes
for a canon WORK_UNIT asset (criticality, nameplate, strategy).';
COMMENT ON COLUMN maint.asset_profile.criticality IS 'RCM criticality band;
drives PM intensity, WO priority, spares stocking.';

```

-- 2. Maintainable-item subtree (ISO 14224 L7-L9 below the WORK_UNIT)

```
CREATE TABLE maint.maintainable_item (  
  item_id      BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,  
  tenant_id    UUID NOT NULL,  
  asset_id     BIGINT NOT NULL REFERENCES  
  canon.equipment_node(node_id), -- owning WORK_UNIT  
  parent_item_id BIGINT REFERENCES maint.maintainable_item(item_id), --  
  self-ref for SUBUNIT→ITEM→PART  
  level        maint.item_level NOT NULL,  
  name         TEXT NOT NULL,  
  iso14224_class TEXT,  
  native_code   TEXT,  
  source_ref    TEXT,  
  ext          JSONB NOT NULL DEFAULT '{}':jsonb  
);  
CREATE INDEX ix_item_asset ON maint.maintainable_item (tenant_id, asset_id);  
CREATE INDEX ix_item_parent ON maint.maintainable_item (parent_item_id);
```

-- 3. Failure-mode library (ISO 14224 reliability language; FMECA fields)

```
CREATE TABLE maint.failure_mode (  
  failure_mode_id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,  
  tenant_id       UUID NOT NULL,  
  asset_id        BIGINT REFERENCES canon.equipment_node(node_id),  
  item_id         BIGINT REFERENCES maint.maintainable_item(item_id),  
  mode_code       TEXT NOT NULL, -- e.g. 'FTS' fail-to-start, 'LEAK', 'VIB'  
  mode_desc       TEXT NOT NULL, -- the effect by which failure is observed  
  mechanism       TEXT, -- physical/chemical process (fatigue, wear, corrosion)  
  cause          TEXT, -- root circumstance (misalignment, lubrication)  
  effect         TEXT, -- consequence on function/plant  
  severity        SMALLINT CHECK (severity BETWEEN 1 AND 10),  
  occurrence       SMALLINT CHECK (occurrence BETWEEN 1 AND 10),  
  detectability    SMALLINT CHECK (detectability BETWEEN 1 AND 10),  
  rpn             INTEGER GENERATED ALWAYS AS (severity * occurrence *  
  detectability) STORED,  
  assigned_strategy maint.maint_strategy, -- RCM decision for this mode  
  detection_method maint.detection_method,  
  ext            JSONB NOT NULL DEFAULT '{}':jsonb  
);  
CREATE INDEX ix_fm_asset ON maint.failure_mode (tenant_id, asset_id);  
COMMENT ON COLUMN maint.failure_mode.rpn IS 'Risk Priority Number =  
severity × occurrence × detectability (FMECA).';
```

-- 4. Meters & meter readings (usage-based PM; TimescaleDB hypertable)

```
-----  
CREATE TABLE maint.meter (  
  meter_id      BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY  
KEY,  
  tenant_id     UUID NOT NULL,  
  asset_id      BIGINT NOT NULL REFERENCES  
  canon.equipment_node(node_id),  
  meter_type    maint.meter_type NOT NULL,  
  uom           TEXT NOT NULL,           -- e.g. 'h', 'cycles', 'MT', 'km'  
  current_value NUMERIC NOT NULL DEFAULT 0, -- cached latest cumulative  
  reading  
  rollover_value NUMERIC,           -- if the meter wraps  
  ext           JSONB NOT NULL DEFAULT '{}'::jsonb,  
  UNIQUE (tenant_id, asset_id, meter_type)  
);
```

```
CREATE TABLE maint.meter_reading (  
  reading_id    BIGINT GENERATED BY DEFAULT AS IDENTITY,  
  tenant_id     UUID NOT NULL,  
  meter_id      BIGINT NOT NULL REFERENCES maint.meter(meter_id),  
  asset_id      BIGINT NOT NULL REFERENCES  
  canon.equipment_node(node_id),  
  reading_ts    TIMESTAMPTZ NOT NULL,  
  reading_value NUMERIC NOT NULL,       -- cumulative meter value at  
  reading_ts  
  delta_value   NUMERIC,           -- increment since previous reading (derived)  
  source_ref    TEXT,              -- PLC tag / manual entry / CMMS  
  PRIMARY KEY (reading_id, reading_ts)  
) PARTITION BY RANGE (reading_ts);  
CREATE INDEX ix_meter_reading ON maint.meter_reading (meter_id, reading_ts  
DESC);  
-- TimescaleDB: SELECT create_hypertable('maint.meter_reading','reading_ts');  
(alternative to native partition)
```

-- 5. Maintenance plans, triggers, task lists (the PM engine model)

```
-----  
CREATE TABLE maint.maintenance_plan (  
  plan_id       BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,  
  tenant_id     UUID NOT NULL,  
  asset_id      BIGINT REFERENCES canon.equipment_node(node_id), -- specific  
  asset ...  
  asset_class   TEXT,           -- ... or a class (template plan)  
  name          TEXT NOT NULL,  
  strategy      maint.maint_strategy NOT NULL DEFAULT 'TIME_BASED',  
  default_priority SMALLINT NOT NULL DEFAULT 3 CHECK (default_priority  
BETWEEN 1 AND 5),  
  est_duration_min INTEGER,  
  requires_shutdown BOOLEAN NOT NULL DEFAULT FALSE,
```

```

    active          BOOLEAN NOT NULL DEFAULT TRUE,
    source_ref      TEXT,
    ext             JSONB NOT NULL DEFAULT '{}':jsonb,
    CHECK (asset_id IS NOT NULL OR asset_class IS NOT NULL)
);

```

```

CREATE TABLE maint.plan_trigger (
    trigger_id      BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    tenant_id       UUID NOT NULL,
    plan_id         BIGINT NOT NULL REFERENCES
maint.maintenance_plan(plan_id) ON DELETE CASCADE,
    trigger_type    maint.trigger_type NOT NULL,
    -- time triggers:
    interval_value  NUMERIC,                -- e.g. 30
    interval_uom    TEXT,                   -- 'DAYS','WEEKS','MONTHS'
    -- meter triggers:
    meter_id        BIGINT REFERENCES maint.meter(meter_id),
    meter_interval  NUMERIC,                -- e.g. every 2000 (in meter uom)
    -- condition triggers:
    condition_tag   TEXT,                   -- sensor tag / meter
    condition_op    TEXT CHECK (condition_op IN ('>','>=','<','<=','=')),
    condition_value NUMERIC,
    -- governance:
    compliance_window_days SMALLINT NOT NULL DEFAULT 7,
    next_due_ts     TIMESTAMPTZ,           -- cached scheduler state
    next_due_meter  NUMERIC,
    last_generated_ts TIMESTAMPTZ,
    ext             JSONB NOT NULL DEFAULT '{}':jsonb
);

```

```

CREATE INDEX ix_trigger_due ON maint.plan_trigger (tenant_id, next_due_ts);

```

```

CREATE TABLE maint.maintenance_task (
    task_id         BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    tenant_id       UUID NOT NULL,
    plan_id         BIGINT NOT NULL REFERENCES
maint.maintenance_plan(plan_id) ON DELETE CASCADE,
    seq            SMALLINT NOT NULL,
    description     TEXT NOT NULL,
    task_type       maint.task_type NOT NULL DEFAULT 'INSPECT',
    est_minutes     INTEGER,
    required_skill  TEXT,
    safety_permit   TEXT,
    spec_low        NUMERIC,                -- acceptable range for a measured task
    spec_high       NUMERIC,
    spec_uom        TEXT,
    ext             JSONB NOT NULL DEFAULT '{}':jsonb
);

```

```

CREATE INDEX ix_task_plan ON maint.maintenance_task (plan_id, seq);

```

```

CREATE TABLE maint.task_part ( -- parts a task expects to consume (task BOM)
    task_part_id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    tenant_id    UUID NOT NULL,
    task_id      BIGINT NOT NULL REFERENCES maint.maintenance_task(task_id)
ON DELETE CASCADE,
    spare_id     BIGINT NOT NULL, -- FK added after spare_part defined
    qty          NUMERIC NOT NULL DEFAULT 1
);

```

-- 6. Notifications (problem reports) & Work Orders (the executed jobs)

```

CREATE TABLE maint.notification (
    notif_id     BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    tenant_id    UUID NOT NULL,
    asset_id     BIGINT NOT NULL REFERENCES
    canon.equipment_node(node_id),
    reported_ts   TIMESTAMPTZ NOT NULL DEFAULT now(),
    reported_by   BIGINT REFERENCES canon.personnel(person_id),
    symptom       TEXT,
    failure_mode_id BIGINT REFERENCES maint.failure_mode(failure_mode_id),
    severity      SMALLINT CHECK (severity BETWEEN 1 AND 5),
    status        maint.notif_status NOT NULL DEFAULT 'OPEN',
    wo_id         BIGINT, -- set when converted (FK added below)
    source_ref    TEXT,
    ext           JSONB NOT NULL DEFAULT '{}'::jsonb
);

```

```

CREATE INDEX ix_notif_open ON maint.notification (tenant_id, status) WHERE
status = 'OPEN';

```

```

CREATE TABLE maint.work_order (
    wo_id        BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    tenant_id    UUID NOT NULL,
    asset_id     BIGINT NOT NULL REFERENCES
    canon.equipment_node(node_id),
    wo_type      maint.wo_type NOT NULL,
    wo_source    maint.wo_source NOT NULL,
    status       maint.wo_status NOT NULL DEFAULT 'PLANNED',
    priority     SMALLINT NOT NULL DEFAULT 3 CHECK (priority BETWEEN 1 AND 5),
    plan_id      BIGINT REFERENCES maint.maintenance_plan(plan_id),
    trigger_id   BIGINT REFERENCES maint.plan_trigger(trigger_id),
    notif_id     BIGINT REFERENCES maint.notification(notif_id),
    failure_event_id BIGINT, -- FK added after failure_event defined
    downtime_event_ref BIGINT REFERENCES canon.event(event_id), -- the SAME canonical downtime row M4 reads
    description  TEXT,
    root_cause   TEXT,

```



```

created_ts    TIMESTAMPTZ NOT NULL DEFAULT now(),
due_ts        TIMESTAMPTZ,           -- for PM compliance window
scheduled_start TIMESTAMPTZ,
scheduled_end  TIMESTAMPTZ,
actual_start   TIMESTAMPTZ,
actual_end     TIMESTAMPTZ,
completed_ts   TIMESTAMPTZ,
closed_ts      TIMESTAMPTZ,
assigned_crew_ref BIGINT,
labor_cost     NUMERIC DEFAULT 0,    -- cached rollups (authoritative lines
below)
part_cost      NUMERIC DEFAULT 0,
downtime_cost  NUMERIC DEFAULT 0,
source_ref     TEXT,
lineage_ref     TEXT,
ext            JSONB NOT NULL DEFAULT '{}':jsonb
);
CREATE INDEX ix_wo_asset ON maint.work_order (tenant_id, asset_id);
CREATE INDEX ix_wo_status ON maint.work_order (tenant_id, status);
CREATE INDEX ix_wo_due ON maint.work_order (tenant_id, due_ts) WHERE
status NOT IN ('CLOSED','CANCELLED');
COMMENT ON COLUMN maint.work_order.downtime_event_ref IS 'Points to the
single canon.event downtime row; M2 classifies it, M4 aggregates it. Never double-
count.';

ALTER TABLE maint.notification
ADD CONSTRAINT fk_notif_wo FOREIGN KEY (wo_id) REFERENCES
maint.work_order(wo_id);

CREATE TABLE maint.wo_task (           -- executed task instances on a WO
wo_task_id    BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY
KEY,
tenant_id     UUID NOT NULL,
wo_id         BIGINT NOT NULL REFERENCES maint.work_order(wo_id) ON
DELETE CASCADE,
task_id       BIGINT REFERENCES maint.maintenance_task(task_id), -- null =
ad-hoc
seq           SMALLINT,
description   TEXT NOT NULL,
reading_value NUMERIC,
reading_uom   TEXT,
result        maint.task_result,
notes        TEXT
);
CREATE INDEX ix_wotask_wo ON maint.wo_task (wo_id);

CREATE TABLE maint.wo_labor (
labor_id      BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
tenant_id     UUID NOT NULL,
wo_id         BIGINT NOT NULL REFERENCES maint.work_order(wo_id) ON

```



```

DELETE CASCADE,
person_ref    BIGINT REFERENCES canon.personnel(person_id),
hours         NUMERIC NOT NULL,
rate_ref      TEXT,                                -- canon.cost_rate code (labour), effective-dated
(D7)
cost         NUMERIC,                            -- hours × rate (resolved at close)
start_ts      TIMESTAMPTZ,
end_ts        TIMESTAMPTZ
);
CREATE INDEX ix_labor_wo ON maint.wo_labor (wo_id);

```

```

-----
-- 7. Failure events (also published to canon.event as failure_event)
-----

```

```

CREATE TABLE maint.failure_event (
  failure_event_id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY
KEY,
  tenant_id       UUID NOT NULL,
  asset_id        BIGINT NOT NULL REFERENCES
  canon.equipment_node(node_id),
  failure_mode_id BIGINT REFERENCES maint.failure_mode(failure_mode_id),
  detection_method maint.detection_method,
  down_ts         TIMESTAMPTZ NOT NULL,
  up_ts           TIMESTAMPTZ,
  downtime_h      NUMERIC GENERATED ALWAYS AS (EXTRACT(EPOCH FROM
(up_ts - down_ts))/3600.0) STORED,
  is_breakdown    BOOLEAN NOT NULL DEFAULT TRUE,
  wo_id           BIGINT REFERENCES maint.work_order(wo_id),
  downtime_event_ref BIGINT REFERENCES canon.event(event_id),
  source          maint.wo_source,                  -- M1_STOPPAGE / capture /
CMMS_IMPORT
  source_ref      TEXT,
  lineage_ref     TEXT
);
CREATE INDEX ix_fe_asset ON maint.failure_event (tenant_id, asset_id, down_ts
DESC);

```

```

ALTER TABLE maint.work_order
  ADD CONSTRAINT fk_wo_failure FOREIGN KEY (failure_event_id)
REFERENCES maint.failure_event(failure_event_id);

```

```

-----
-- 8. Spares / MRO inventory & movements
-----

```

```

CREATE TABLE maint.spare_part (
  spare_id       BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY
KEY,
  tenant_id      UUID NOT NULL,
  part_no        TEXT NOT NULL,
  description     TEXT,

```

```

uom          TEXT NOT NULL DEFAULT 'EA',
unit_cost    NUMERIC,
on_hand_qty   NUMERIC NOT NULL DEFAULT 0,
reserved_qty  NUMERIC NOT NULL DEFAULT 0,
reorder_point NUMERIC,
reorder_qty    NUMERIC,
lead_time_days SMALLINT,
criticality    maint.criticality,
bin_location  TEXT,
source_ref    TEXT,
ext           JSONB NOT NULL DEFAULT '{}':jsonb,
UNIQUE (tenant_id, part_no)
);

```

```

ALTER TABLE maint.task_part
ADD CONSTRAINT fk_taskpart_spare FOREIGN KEY (spare_id) REFERENCES
maint.spare_part(spare_id);

```

```

CREATE TABLE maint.spare_bom (           -- which spares serve which
item/asset
  bom_id      BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY
KEY,
  tenant_id   UUID NOT NULL,
  spare_id    BIGINT NOT NULL REFERENCES maint.spare_part(spare_id),
  item_id     BIGINT REFERENCES maint.maintainable_item(item_id),
  asset_id    BIGINT REFERENCES canon.equipment_node(node_id),
  qty_per     NUMERIC NOT NULL DEFAULT 1,
  CHECK (item_id IS NOT NULL OR asset_id IS NOT NULL)
);

```

```

CREATE TABLE maint.wo_part (           -- parts consumed by a work order
  wo_part_id  BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY
KEY,
  tenant_id   UUID NOT NULL,
  wo_id       BIGINT NOT NULL REFERENCES maint.work_order(wo_id) ON
DELETE CASCADE,
  spare_id    BIGINT NOT NULL REFERENCES maint.spare_part(spare_id),
  qty         NUMERIC NOT NULL,
  unit_cost   NUMERIC,
  cost        NUMERIC,           -- qty × unit_cost
  issued_ts   TIMESTAMPTZ DEFAULT now()
);
CREATE INDEX ix_wo_part_wo ON maint.wo_part (wo_id);

```

```

CREATE TABLE maint.spare_txn (         -- stock ledger
  txn_id      BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  tenant_id   UUID NOT NULL,
  spare_id    BIGINT NOT NULL REFERENCES maint.spare_part(spare_id),
  wo_id       BIGINT REFERENCES maint.work_order(wo_id),

```

```

txn_type      maint.spare_txn_type NOT NULL,
qty           NUMERIC NOT NULL,
txn_ts        TIMESTAMPTZ NOT NULL DEFAULT now(),
balance_after NUMERIC
);
CREATE INDEX ix_sparetxn_spare ON maint.spare_txn (spare_id, txn_ts DESC);

```

-- 9. Condition alerts (basic CBM v1) & predictive hooks (build deferred)

```

CREATE TABLE maint.condition_alert (
  alert_id      BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  tenant_id     UUID NOT NULL,
  asset_id      BIGINT NOT NULL REFERENCES
  canon.equipment_node(node_id),
  trigger_id    BIGINT REFERENCES maint.plan_trigger(trigger_id),
  raised_ts     TIMESTAMPTZ NOT NULL DEFAULT now(),
  tag           TEXT,
  observed_value NUMERIC,
  threshold_value NUMERIC,
  severity      SMALLINT CHECK (severity BETWEEN 1 AND 5),
  status        TEXT NOT NULL DEFAULT 'OPEN',
  wo_id         BIGINT REFERENCES maint.work_order(wo_id)
);

```

-- Predictive (PdM) is FUTURE SCOPE — see 08_Predictive spec. The table below is the

-- forward-compatible health/RUL output slot; no producer is built in v1.

CREATE TABLE maint.asset_health (*-- written by future OSA-CBM service; empty in v1*

```

  health_id     BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  tenant_id     UUID NOT NULL,
  asset_id      BIGINT NOT NULL REFERENCES
  canon.equipment_node(node_id),
  assessed_ts   TIMESTAMPTZ NOT NULL,
  health_grade  SMALLINT,           -- 0-100
  anomaly_score NUMERIC,
  rul_hours     NUMERIC,           -- remaining useful life forecast
  model_ref     TEXT,              -- MLflow model id (future)
  advisory      TEXT
);

```

COMMENT ON TABLE maint.asset_health **IS** 'FUTURE-SCOPE output of the OSA-CBM/prognostics pipeline (RUL, health). No v1 producer.';

-- 10. KPI daily rollup (materialized to the serving/mart layer; example grain)

```

CREATE TABLE maint.kpi_asset_daily (
  tenant_id     UUID NOT NULL,

```

```

    asset_id      BIGINT NOT NULL REFERENCES
canon.equipment_node(node_id),
    kpi_date      DATE NOT NULL,
    failures      INTEGER DEFAULT 0,
    downtime_h    NUMERIC DEFAULT 0,
    repair_h      NUMERIC DEFAULT 0,
    operating_h   NUMERIC DEFAULT 0,
    mtbf_h        NUMERIC,
    mttr_h        NUMERIC,
    availability   NUMERIC,           -- MTBF/(MTBF+MTTR), inherent
    pm_scheduled  INTEGER DEFAULT 0,
    pm_completed  INTEGER DEFAULT 0,
    pm_compliance NUMERIC,
    maint_cost    NUMERIC DEFAULT 0,
    downtime_cost NUMERIC DEFAULT 0,
    PRIMARY KEY (tenant_id, asset_id, kpi_date)
);

--
=====
-- End M2 reference schema. Migrations via Flyway/Alembic (additive, versioned).
--
=====
=====

```